Paidly — Product Architecture Strategy & Blueprint

Purpose: Shift from a *page list* mental model to **systems** and a defined **core engine**, while keeping the technical map honest.

PAIDLY ARCHITECTURE V2 (CLEAN REFERENCE)

*One-page upgrade summary. This section is the canonical **Paidly v2 system map** used across product and engineering.*

Paidly v2 Systems (final):

1. **Identity System** — Auth, users, organizations, roles, RLS-backed tenancy.
2. **Document Engine** — Shared compose, send, and PDF behaviour for commercial documents (invoices, quotes, payslips) **plus** a Documents Hub for other business document types. Persistence is split: specialised tables vs `documents`.
3. **Payment Intent Layer** — Canonical handoff from document delivery/observation to payment rails and settlement orchestration.
4. **Revenue System** — Payment providers, subscriptions, cash flow, and reporting (reads/rails downstream of documents).
5. **Relationship System** — Clients + catalog + offering intelligence that feeds document composition.
6. **Experience System** — Shared UI/interaction contracts (shell, sticky actions, money UX consistency).
7. **Payment Intelligence Layer** — Get Paid logic: reminders, nudges, triggers, and follow-up intelligence.

Frontend layout system (consistent everywhere):

- **Header** — title + primary **actions**
- **Content** — tables, grids, main **forms**
- **Side panel** — **summary**, filters, secondary controls (`PageTemplate` in code)

Data flow (client):

```
UI → Hooks → Services → Entity facades (EntityManager) → Supabase
```

**Serverless APIs (`/api/`)* handle:

- Payments (**Payfast**)
- Emails (**Resend** / SMTP)
- Public document access (shares, tokens)
- **Cron** jobs (dunning, reminders, ops)

Deployment note: Same-origin `/api` on Vercel is implemented by ** `api/*.js` *serverless functions**; **the optional** *Express **app in** `server/src/index.js` **is a** separate **runtime (rate limits and middleware apply only where that process fronts** `/api`**). See** `docs/API_DEPLOYMENT_MODEL.md`* before tuning limits or tracing 429s.

Goal: Evolve Paidly from an **invoicing tool** into a full **business operating system**—same story as **Positioning** below.

On-call session/runtime guardrails: see `docs/SESSION_RUNTIME_GUARDS.md`.

New to the codebase: see **`docs/HOW_EVERYTHING_CONNECTS.md`** (one-page diagrams: browser ↔ Supabase ↔ `/api` ↔ optional Express).

Runtime scalability & coordination: `docs/runtime-audit-report.md`, `src/core/` (`RuntimeCoordinator`, `RealtimeManager` logical registry over `paidlyRealtimeManager`, query policies, mutation dedupe, request budgeting, error classification).

Marketing SEO / structured data: homepage JSON-LD (`Organization`, `WebSite`, `WebApplication` offers) follows Google structured data policies — visible content only, no fabricated ratings. See `docs/SEO_STRUCTURED_DATA.md`.

POSITIONING — REAL PRODUCT ADVANTAGE

Paidly is not an invoicing app. It is a **business operating system** for SMBs: one place to **issue** commercial documents, **run** client and catalog relationships, **see** money and performance, and **grow** through channels like affiliates—without treating each area as a separate product bolted together in the UI only.

Investor-facing framing: shift the story from "invoice app" to **financial workflow platform for SMEs**.

You already have (in shipped or advanced form): invoices, quotes, payslips (specialised commercial tables), Documents Hub (generic business documents), clients, reporting, affiliates, and the plumbing for payments, subscriptions, and cash visibility.

Competitive edge: most tools **excel at one slice** (invoicing-only, or accounting-only, or a disconnected referral program). **Few competitors unify** issuance (document engine), relationship/catalog input, revenue/ops read models, and payment intelligence **under one coherent architecture** and vocabulary. That unification—**Document → Payment Intent → Revenue System** plus shared **Experience** and **Payment Intelligence**—is the defensible story: not "another invoice PDF," but **operating the business** in one system.

Execution reality: roughly 70% of the SaaS architecture is already in place. The remaining value-defining 30% is: payment abstraction, event tracking, and experience consistency.

EXECUTIVE FRAMING

Old lens	New lens
"Invoicing app" / feature checklist	**Business operating system** — documents + relationships + money + growth
"We have Invoices, Quotes, Payslips…"	**One Document Engine** (shared compose/send/PDF) with **split persistence**
"Invoices page, Quotes page, Clients page…"	Commercial lists over specialised tables; Documents Hub for other business documents
Features as separate products	**Five core systems** + one **Experience system** (cross-cutting)
Routes drive architecture	**Capabilities** drive architecture; routes are just entry points

Brutal truth the strategy fixes: the stack was described accurately but not **product-driven**: no named "engine," no explicit system boundaries, and no place for **scaled UX consistency** beyond ad hoc pages.

Part A — Product architecture (systems)

A.1 PAIDLY V2 SYSTEMS (FINAL ARCHITECTURE)

These are the **real** architecture—not the sidebar.

1. Identity System

Job: Who is acting, for which organisation, with what authority.

- Supabase Auth (sessions, JWT)
- `profiles`, `organizations`, `memberships`, roles (`admin`, `management`, …)
- RLS as the enforcement layer; client as the UX layer

Organisation vs company/brand vs team (do not confuse these):

Concept	Meaning	Persistence
Organization / account	The Paidly tenant using the product	`organizations`; `CompanyContext.companyId` is this **org id** (product copy often says `company_id` for the tenant)

Concept	Meaning	Persistence
Company / brand	A trading identity that belongs to that organization, used for document branding	`public.companies` (`org_id` , `name` , `logo_url`); invoices set **`invoices.company_id` → `companies.id`**
Team / membership	People who have access to the organization	`memberships` + org roles

Do not create a second tenant system. `CompanyContext` remains org RBAC. Active brand is a **UI default for new documents** (per-org localStorage); changing it must not rewrite existing invoices. Quotes have no `company_id` column (name/logo snapshot only). Payslips stay organization-profile scoped. See `docs/MULTIBRAND_COMPANIES.md` .

Strategy: Stabilise, don't expand. Harden session edge cases, org bootstrap, and role gates (`RequireAuth`) so every other system trusts Identity cheaply.

Auth + session stabilization (in flight): one coherent story for **session read/write** (`getStableSession` / SessionCoordinator vs reserved raw `getSession` in AuthContext / SupabaseAuthService / supabaseAuthRefresh), **profile restore** when local auth cache is cleared, **invite / password / org bootstrap** flows, and **no silent "logged in but empty user"** states. Client work touches **`customClient.js`** / `AuthManager` , **`RequireAuth`** , and Supabase Auth config—treat this as **foundation** before shipping net-new document features at scale.

Connection lifecycle (client OS): `ConnectionLifecycleManager` (`src/lib/connection/ConnectionLifecycleManager.js`) is the single ingress for **auth, refresh, realtime, visibility, network, sleep/wake, and recovery** signals; it updates a read model store and delegates terminal decisions to `SessionOrchestrator` only through one adapter (`registerSessionAuthority(...toSessionAuthorityAdapter())`). Subsystems **report in**; they do not apply parallel session health policies. Transport events use semantic types (`LifecycleSignalType` , e.g. `REALTIME_DISCONNECTED` , `REFRESH_SKIPPED`) and **`lifecyclePolicy`** chooses effects (ignore vs recover vs reconnect) so realtime/visibility noise does not escalate to logout. The same signals also feed **`RuntimeCoordinator`** (`runtimeCoordinatorBridge.js`) for **phase + `pauseNonCriticalRequests`** so authenticated HTTP (`apiRequest` / `backendApi`) waits during AUTH_RECOVERING / RECONNECTING. **Session health** uses a degraded ladder (`sessionRecoveryEscalation.js` + `SESSION_STATUS` : `unstable` → `reconnecting` → `degraded` → `reauth_required`) before `transitionToExpired` for recoverable failures (reconnect, wake, reauth-class 401s); fatal refresh / explicit sign-out still expire immediately.

Sleep/wake app recovery lock: **`AppRecoveryLock`** (`src/lib/recovery/appRecoveryLock.js`) + **`wakeRecoveryStore`** enforce phased recovery: **`wakeRecoveryGuard`** blocks **`customClient`** mutations, **`SyncEngine`** skips queue draining while `blockMutations` is set, **`paidlyRealtimeManager`** suppresses postgres_change delivery during the lock, and **`runWakeRecoverySequence`** restores the JWT/session first (`lockPhase: auth`), then **`awaitRealtimeRecoveryHandlers`** + **`waitForPaidlyMainChannelJoined`** (`lockPhase: realtime`) before unlocking the UI.

Realtime + JWT rotation: On every successful refresh (`TOKEN_REFRESHED` , **`refreshSession`** success, optional **`USER_UPDATED`**), call **`reconcilePaidlyRealtimeAfterTokenRefresh`** (`paidlyRealtimeManager.js`): **`supabase.realtime.setAuth(access_token)`** , then **fully remove** the multiplex channel (**`removeChannel`**) and **recreate** it so postgres_changes always bind to the fresh JWT (no "joined but stale auth" shortcuts). Rebuild debouncing, heartbeat stale detection, transport error backoff, and a small **`RealtimeConnectionPhase`** state machine (**IDLE / CONNECTING / CONNECTED / STALE / RECONNECTING / FAILED**) live in **`paidlyRealtimeManager`** + **`paidlyRealtimeConnectionMachine.js`** ; **`ConnectionMonitor`** maps UX only (no parallel realtime reconnect timers). Visibility alone does **not** run a dedicated "stale realtime repair" path — recovery is heartbeat / transport / auth–driven. Use **`validatePaidlyRealtime()`** for a cheap health snapshot.

Background session resync: **`sessionRefreshScheduler`** (`src/lib/session/sessionRefreshScheduler.js`) is the single coalescing entry for **refresh + profile + realtime hint** work driven by visibility, online, heartbeat, keep-alive, tab sync, wake follow-ups, etc.; initiators call **`requestSessionRefresh({ source })`** instead of invoking **`refreshSession`** directly (the executor is registered from **`AuthContext.impl.jsx`**).

---

2. Document Engine (the commerce core)

Job: Everything a business **issues to another party** to record obligation, intent, or compensation—then **delivers**, **tracks**, and often **collects** on.

Canonical product vocabulary (feature → source of truth):

Feature	Persistence (system of record)
Invoice	`invoices`
Quote	`quotes`
Payslip	`payslips`
Recurring invoice	`recurring_invoices`
Other business documents (leave, expenses, contracts, …)	`documents` (+ `document_items`, `document_events`)

Do not dual-write. Invoices, quotes, payslips, and recurring invoices must not be copied into `documents`. The Documents Hub is not a second store for commercial documents.

Persistence: commercial documents remain on specialised Supabase tables. Shared UI and helpers live in `src/document-engine/`. Ownership policy: `src/document-engine/documentSystemOfRecord.js`.

Migration status (implementation):

- **Shared now (code-level contracts, split persistence):**
- Shared document type vocabulary and routing helpers in `src/document-engine/`.
- Shared list-controller and adapter pattern for invoice/quote/payslip list screens.
- Shared pagination and shared export assembly service paths.
- **Documents Hub (generic types only):**
- Leave requests, expense claims, contracts, and other catalog types persist in `public.documents`.
- Hub **Convert to invoice / quote** (job cards, reports, proposals, scopes) opens specialised compose (`CreateDocument/invoice|quote?fromHubDocument=`) and writes to `invoices` / `quotes`. It does not insert commercial types into `documents`.
- Quote → invoice remains on the Quotes page (`CreateInvoice?quoteId=` / `CreateDocument/invoice?quoteId=`).
- Leftover `documents` rows with `type=invoice|quote|payslip` (if any exist from earlier experiments) are **hidden from the hub list**. Opening a direct URL shows a cleanup screen: go to the specialised list, or archive/remove the leftover hub row. Rows are not migrated and not auto-deleted.
- **Not a goal:**
- Migrating invoices, quotes, or payslips into `documents`.
- Dual-write / automatic mirroring of commercial documents into the hub.

Reframe: one **engine** (compose, send, PDF) with **two persistence owners**:

Document kind	Today's surface	Same engine primitives
Invoice	Invoices, recurring	Draft → send → view/track → pay / remind
Quote	Quotes, templates	Draft → send → accept/expire → convert
Payslip	Payslips	Draft → send → employee view

Shared lifecycle (conceptual):

`Author` → `Compose` (lines, tax, branding) → `Render` (PDF/HTML) → `Deliver` (email, link, portal) → `Observe` (opens, reminders, delivery/engagement telemetry) → `Payment Intent` (amount/currency/expiry + rail handoff) → `Settle` (payment, acceptance, archive)

Observe layer (formalized)

`Observe` is not a loose log. It is a first-class event layer that records delivery, engagement, and money-adjacent milestones in a single stream per document.

Schema upgrade (`document_events`):

- `id`
- `document_id`
- `event_type` (`sent` | `opened` | `clicked` | `paid` | `reminded`)
- `occurred_at`
- `actor_type` (system | recipient | user | webhook)
- `metadata` (jsonb for channel, provider payload refs, reminder run id, etc.)

Event taxonomy (minimum canonical set):

- `sent`
- `opened`
- `clicked`
- `paid`
- `reminded`

This powers:

- Timeline (client and document history)
- Notifications (state-aware in-app/email nudges)
- Smart reminders (behavior + due-date + payment-intent aware)
- Analytics (delivery-to-payment funnel and conversion diagnostics)

Why this matters for product:

- One **mental model** for PM, design, and eng.
- One place to invest: **send pipeline**, **PDF pipeline**, **public token model**, **status vocabulary**, **line-item model**.
- One observable event stream (`document_events`) that unifies communication and payment lifecycle telemetry.
- Feature parity (e.g. quote send = invoice send) becomes **engine work**, not three copies.

Technical anchor today: `Invoice` / `Quote` / `Payslip` entities + `InvoiceSendService` -style orchestration + `/api/send-email` + public share routes. **In code:** `src/document-engine/` exports `DOCUMENT_TYPES` , `normalizeDocumentType` , `parseRouteDocumentTypeStrict` , `getDocumentEntity` , `documentRef` —use these for routing, analytics, and new engine code. **Roadmap:** grow this module (shared send/PDF adapters, shared status vocabulary) so new document kinds plug in, not fork.

Critical addition: Payment Intent layer (Document Engine ↔ Revenue & Ops)

Without a first-class `Payment Intent` , payments feel bolted on, Payfast-specific logic leaks across product surfaces, and the engine is harder to scale. Introduce a canonical handoff object between document delivery/observation and financial capture.

Why this matters:

- **Without it:** payments feel bolted on, provider logic leaks everywhere, and cross-surface consistency breaks.
- **With it:** clean abstraction between document and payment rails, multi-provider readiness, and analytics-ready payment funneling.

Payment intent contract (per payable document):

- `intent_id` (idempotent)
- `document_ref` (`type` , `id` , `org_id`)
- `amount_snapshot` + `currency_snapshot`
- `payer_context` (public share, portal user, or signed-in actor)
- `rail` (`payfast` , future rails), `expires_at`
- `status` (`pending` , `requires_action` , `paid` , `failed` , `cancelled` , `expired` , `refunded`)

Schema upgrade (`payment_intents`):

- `id`
- `document_id`
- `provider` (`payfast` | `stripe`)
- `amount`
- `currency`
- `status`
- `external_id`
- `created_at`

Boundary of ownership:

- **Document Engine owns:** payable snapshot creation, due/expiry semantics, and `document_ref` identity.
- **Revenue & Ops owns:** provider orchestration, webhook verification, settlement/reconciliation, and ledger-like payment records.
- **Experience System owns:** one payment-status language and CTA behavior across document detail, public links, and portal views.

Result: no direct "mark paid" shortcuts from UI paths; all payable settlement flows through `Payment Intent` + verified payment events.

Product upgrade: one compose surface, many kinds

The **big upgrade** is not more list pages—it is **one mental journey** with honest persistence:

1. **Create** from a consistent compose pattern (brand, line items, preview).
2. **Commercial types** (invoice / quote / payslip) write to specialised tables; **other types** write to the Documents Hub.
3. **Shared UI** where it already exists — editors, preview, send affordances (Experience System + templates).
4. **Different logic** — status machines, tax/settlement rules, payroll vs AR: **kind-specific adapters** behind the same surface.

List routes ("Invoices", "Quotes", "Payslips") remain **indexes** over their specialised tables. The Documents Hub lists generic `documents` rows only.

---

5. Relationship System

Job: Who you sell to and **what** you sell—data that **feeds** the Document Engine.

- **Clients** (CRM): contact, terms, portal access
- **Catalog** (`services`): products & services, pricing, inventory where relevant
- **Line items** on documents: snapshots + links back to catalog when useful

Strategy: Treat this as the **input graph** to commerce—not a separate "Contacts app." List screens are views; the system is **relationship + SKU/rate intelligence**. Growth-oriented **CRM behaviour** (follow-ups, nudges, sequences) lives primarily in the **Growth Engine**; this system owns **master data** and what gets embedded on documents.

---

4. Revenue System (Revenue & Ops)

Job: Everything that turns **issued documents** (especially invoices) into **cash, visibility, and tenant billing**—without re-implementing document authoring here.

- **Payments:** Payfast, invoice payment state, webhooks (`api/payfast-handler`)
- **POS integrations:** Generic webhook + Yoco/Square ingress (`api/pos/*`), normalized `pos_sales_events`, optional catalog stock decrement via `adjust_inventory_stock` (source `pos`). **Connect flows:** Square OAuth (`/api/pos/oauth/square/start` → callback); Yoco secure connect (API secret → auto webhook registration). Settings → Integrations; dashboard **POS sales today** card.
- **Payment intents:** capture/retry/expiry orchestration and provider status normalization
- **Cash flow:** timelines and balances built from documents + payments

- **Reports:** read models and exports on top of documents + payments + catalog where relevant
- **Subscriptions & dunning:** how Paidly bills the customer (packaging, crons in `vercel.json` → `/api/cron/...`)
- **SaaS subscription SoR (billing v2):** `plans` catalog (`amount`, `billing_cycle`, `plan_family`, `payfast_item_name`, `features`, `active` — **not hardcoded in React**) + public tiers Starter R50 / Business R150 / Growth R350 (+ annual 2 months free) + Enterprise contact-sales; legacy Individual/SME/Corporate grandfathered. `subscriptions` core (`company_id`, `plan_id`, `plan_family`, `grace_ends_at`, PayFast ids, period bounds, `cancelled_at`, `created_by`, `trial_started_at` / `trial_ends_at`, `subscription_source` (`system_trial` \| `payfast` \| `admin`), `admin_override`; status allow-list only: `pending` \| `processing` \| `active` \| `past_due` \| `failed` \| `cancelled` \| `expired` \| `suspended` \| `trialing`) + append-only `payment_history` + `subscription_events` + `payfast_itn_logs` + `subscription_invoices` + `webhook_logs`. **New org-owner accounts created on/after 2026-08-20 00:00:00 UTC** get a **7-day server-side trial** (`trialing`); invited members do not. Access is `hasSubscriptionAccess` (trial not expired, `active`, or admin-managed). Activation to paid only after verified PayFast ITN (never from the SPA). Admin PATCH `/api/admin/subscriptions` can extend/grant/suspend/cancel and always sets `admin_override` so expiry jobs cannot revert it; changes write `audit_logs`. **Payment/revenue reporting** counts only `payment_history.payment_status = completed` with transaction/created time **≥ 2026-08-20T00:00:00.000Z** (UTC) — not trials, admin grants, pending/failed, or pre-cutoff rows. **APIs:** `GET /api/subscriptions/plans`, `POST /api/subscriptions/create|change|cancel`, `GET …/status|current`, `POST /api/payfast/itn`, `GET /api/admin/subscriptions` (overview + reporting), `GET /api/admin/payments`, `PATCH /api/admin/subscriptions` (legacy client-priced `POST /api/payfast/subscription` returns 410). Schema: `docs/SUBSCRIPTION_BILLING_SCHEMA.md`. Profiles (`plan` / `subscription_status` / `is_pro`) remain a cache (mirrored as `plan_family`).

Feeds off the Document Engine: revenue truth is **downstream of document state** (totals, status, due dates, line items). Revenue & Ops aggregates, reconciles, and projects; it does not fork "another invoice."

Strategy: Keep business rules that define **what a document is** (e.g. when an invoice is *overdue* in product terms) aligned with the Document Engine; keep **money rails** (capture, allocate, subscription charge) here.

Payment integration rule: provider events never mutate document totals directly. They update payment-intent/payment records first; document settlement status is derived via verified reconciliation rules. SaaS tenant billing is separate from invoice `payments`: ITN must verify signature, PayFast server confirm, amount, merchant, and subscription correlation before moving `subscriptions` off `pending`.

SaaS billing security principles (non-negotiable): (1) Never trust the frontend for payment state. (2) Verify every ITN server-side with PayFast before changing subscription status. (3) Database is SoR. (4) Log every lifecycle event and webhook. (5) Idempotent webhooks (no duplicate activations). (6) RLS everywhere; service role only where required. (7) Never expose merchant credentials, signatures, or verification logic to the client. (8) Validate merchant ID, amount, currency, and subscription identifiers before activate. (9) Prefer atomic DB transactions for payment history + subscription update + event logs. Details: `docs/SUBSCRIPTION_BILLING_SCHEMA.md`.

Responsibility split (hard boundary)

Document Engine owns:

- Document status semantics (`paid`, `overdue`, lifecycle transitions)
- Document totals and payable snapshots

Revenue & Ops owns:

- Payment providers (Payfast, Stripe, future rails)
- Subscriptions and dunning/billing operations
- Cash flow models
- Reporting/read models

Boundary outcome: clear separation keeps document rules coherent while allowing payment/billing systems to scale independently.

7. Payment Intelligence Layer (Get Paid System)

Job: Turn events + payment intent state into proactive actions that improve collection velocity.

- Inputs: `document_events`, `payment_intents`, due dates, client/payment history
- Decisions: reminder timing, CTA sequencing, retry windows, provider fallback policy
- Outputs: auto reminders, smart nudges, “invoice viewed but not paid” triggers, suggested follow-ups, prioritized “at-risk” invoices, and payment funnel analytics

Example trigger (v1):

Client viewed invoice 3 times without a `paid` event in the observation window → trigger reminder and surface a suggested follow-up action in the right panel.

Position in architecture: sits between observation/payment state and user-facing actions, orchestrating how Paidly gets customers paid faster without leaking provider logic into page code.

Growth loops (implemented within Revenue, Relationship, and Experience systems)

Job: How Paidly **acquires, retains, and scales**—loops that sit beside day-to-day issuing.

- **Affiliates:** acquisition + admin moderation (`api/affiliates`, …)
- **Emails:** transactional and campaign-style sends from `/api` and app-triggered flows
- **Notifications:** in-app + scheduled nudges (crons, reminders, due-date services)
- **CRM behaviour:** follow-ups, client engagement, portal nudges—**behaviour** on top of Relationship data, not a duplicate CRM product

This is how Paidly scales: repeatable growth mechanics without entangling them in the document compose path.

Operator / platform admin (users, oversight, platform messages, `/admin-v2/*`) can be documented as part of this engine or Identity, depending on audience—treat it as **platform operations**, not SMB document logic.

---

6. EXPERIENCE SYSTEM (UX AT SCALE)

Job: So Paidly **feels** like one product, not twenty features stitched together.

Pillars:

Pillar	Meaning
Shell	Repeated editor layout: full-width work area, `max-w-*` content, sticky summary/actions (pattern already emerging: EditQuote, EditClient, EditCatalogItem).
Tokens	`border-border`, `bg-card`, `text-muted-foreground`—no one-off palette per page.
Data discipline	React Query keys, invalidation rules, **cache-first navigation** (`paidlyClientCachePolicy.js` + `docs/Paidly-Caching-Architecture.md`). Hydrate from Zustand/Dexie/RQ then refresh when stale — not a full cold load on every route entry. **Observability:** `paidlyDataLayerInstrumentation.js` (cache restore, dedupe, realtime patches) + `[PaidlyRealtime]` structured logs (reconnect/backoff/cooldown).
A11y & forms	Label/`id` parity, focus order, disabled states that explain *why* (title/tooltip).
Money UX contract	Same status chips + primary CTA logic (`Pay now`, `Retry payment`, `View receipt`) everywhere a payable document appears.
Page template	List/index pages share one **three-zone** shell (below)—visual consistency reads as **premium**.

Strategy: Treat “Experience” as **governed**: a short **layout + form checklist** for any new surface that touches Identity or the Document Engine—same as you’d gate API changes.

Experience upgrade: make the editor pattern universal

The editor shell is a product advantage only if it is consistent across **documents and money actions**. Standardize the same right-side sticky panel pattern for invoice payment actions, not just compose/edit forms.

Universal right panel contract (sticky):

- Primary amount block (`Total`, optional secondary currency conversion).
- Primary action first (`Pay Now` when payable).
- Secondary lifecycle actions below (`Send Reminder`, receipt/share actions by state).
- Deterministic state switching from `Payment Intent` status (no page-specific CTA ordering).

Invoice payment panel (reference pattern):

- `Total: $100`
- `≈ R1,638`
- `[ Pay Now ]`
- `[ Send Reminder ]`

Page Template System (UI system — critical)

The UI is improving but must be **systemized**, not page-by-page improvisation.

Every primary list / index page follows three zones:

1. **Header** — page title, one-line purpose, **primary actions** (create, import, refresh, layout toggle). Prefer `PageHeader` (`src/components/dashboard/PageHeader.jsx`) inside `PageTemplate.Header`.
2. **Content** — main **table or grid** (or tabbed main). This is the scroll-heavy region; keep widths `min-w-0` so tables don't blow the shell.
3. **Side panel** — **filters**, **counts / summary**, secondary controls. On large screens: **sticky** column (`lg:`) to the right of content; on small screens: stack **below** main or **above** the table (pick one per product area and stay consistent).

Code: `PageTemplate` + `PageTemplate.Header` + `PageTemplate.Body` (`sidePanel` prop) in **`src/components/layout/PageTemplate.jsx`**. Import from `@/components/layout` or `@/components/layout/PageTemplate`. Use **`embedded`** when the page already sits inside a padded shell (e.g. **`AdminLayout`**).

Conformance targets (same chrome, same rhythm):

Page	Scope
Invoices	Primary document list — header/actions + table; filters/summary → side panel
Quotes	Same pattern as invoices
Clients	Relationship index — header + list/grid; filters or segment summary → side
Services	Catalog / inventory — header + content; industry/templates/search → side where possible
Affiliates	Growth admin — `PageHeader` already; align body to **content + sticky side** for search/status

Refactors can be **incremental**: introduce `PageTemplate` first, then move filters/summary into `sidePanel` per page without changing data logic.

Standardize UI layout (everything same structure)

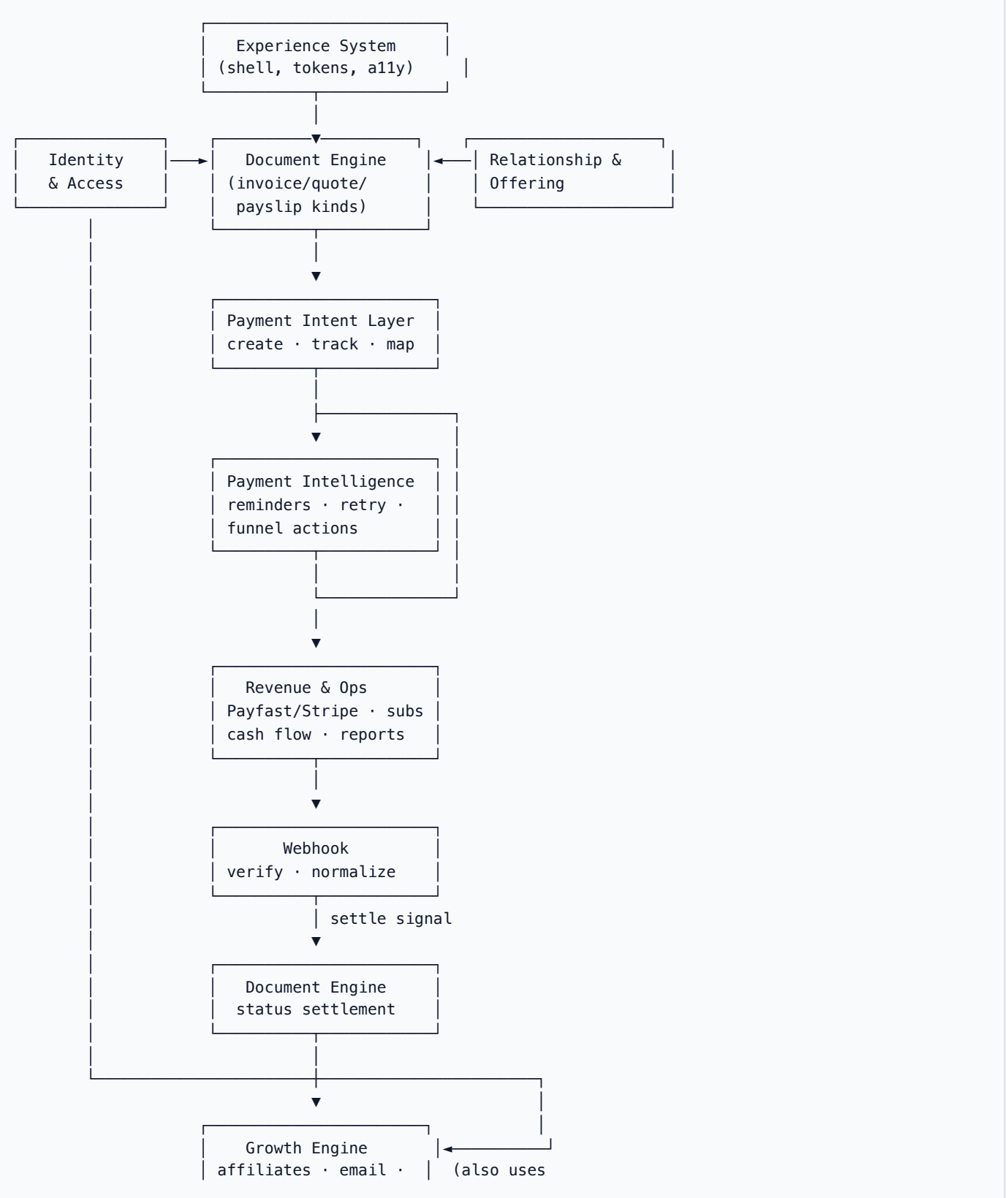
Rule: major surfaces share one **structural grammar** so the product reads as intentional, not a stack of one-off pages.

Surface type	Structure
List / index (Invoices, Quotes, Clients, Services, Affiliates, …)	**`PageTemplate`**: Header (title + actions) → Content (table/grid) → Side panel (filters / summary)
Document compose / edit	Shared **editor shell**: full-width work area, `max-w-*` content, **sticky** summary + primary actions (align `EditInvoice` / `EditQuote` / `CreateDocument` patterns)
Payment touchpoints (document detail, public invoice, portal)	Shared payment state model driven by `Payment Intent` status; same CTA order and error/retry messaging

Surface type	Structure
Admin / settings-style	`PageHeader` + main column; use `PageTemplate` **with** `embedded` when inside `AdminLayout` if a side rail helps

Deliverable: keep the **Experience checklist** short: “Which template? Header actions? Side panel? Sticky save?”— **every new page picks a row**, no ad hoc fourth layout.

A.3 SYSTEM MAP (ONE PICTURE)



```
                    │ notifications · CRM    │   relationship data)
                    └────────────────────────┘
```

Money loop (explicit):

`Document Engine` → `Payment Intent Layer` → `Revenue & Ops` (Payfast/Stripe) → `Webhook` → `Document Engine (Settle)`

Get Paid loop (differentiator):

`Observe` (`document_events`) + `Payment Intent` state → `Payment Intelligence Layer` → smart reminders/CTA/retry strategy → `Revenue & Ops` + `Experience System`

Part B — Technical blueprint (stack & flow)

B.1 WHAT PAIDLY IS (PRODUCT ONE-LINER)

Paidly is a **business operating system for SMBs**—not a narrow invoicing tool: **documents** (invoice / quote / payslip), **relationships & catalog**, **revenue visibility & payments**, and **growth** (e.g. affiliates) in one product story. **South Africa–first** (Payfast, ZAR defaults). **Stack: Vite + React** SPA on **Vercel**, **Supabase** as system of record.

B.2 WHAT RUNS WHERE

Layer	Technology	Role
Frontend	React 18, Vite 6, React Router 7	UI; lazy routes
Styling	Tailwind, Radix, Framer Motion	Components + motion
State	TanStack Query, Zustand, Context	Cache, auth shell, prefs
Data	Supabase JS	Postgres, Auth, RLS
APIs	Vercel `/api/*`	Email, shares, Payfast, crons, affiliates
Optional	`server/` Express	PDF/email dev tooling
Analytics	`@vercel/analytics`	Prod only

B.3 EXTERNAL SERVICES

Service	Role
Supabase	DB + Auth (`VITE_SUPABASE_*`)
Vercel	Host + serverless + crons + redirects
Payfast	Payments / subscriptions
Resend / SMTP	Transactional email from `/api`
Anvil	PDF generation paths (tooling + app)
IP rate limits	Sign-in / sign-up / forgot-password via Node auth API when enabled

B.4 REPOSITORY MAP

- `src/pages/` — route entrypoints (thin controllers)
- `src/components/layout/PageTemplate.jsx` — **Page Template System** (header + body grid + optional sticky side panel); `embedded` for `AdminLayout`
- `src/components/`, `src/services/` — feature + domain logic; **list/query orchestration** lives in `src/services/*` (called from hooks), not in pages
- `src/api/customClient.js` — `EntityManager` → Supabase (+ localStorage for unmigrated entities)

- `api/` — Vercel handlers
- `supabase/migrations/` — schema

See **`docs/SUPABASE_DATA_MODEL.md`** for table ↔ entity detail.

For runtime ownership and boundary rules (Browser/Supabase/Edge-Server), see **`docs/ARCHITECTURE_RUNTIME_MAP.md`**.

For the connection lifecycle coordinator, semantic events, and authority rules, see **`docs/CONNECTION_LIFECYCLE_ARCHITECTURE.md`**.

B.5 DATA FLOW (REFINED)

Canonical client stack

Do **not** wire pages or heavy components **straight to** `Invoice`, `Quote`, etc. for anything beyond trivial one-offs. Prefer:

```
UI (pages / components)
  → Hooks (TanStack Query, local UI state)
    → Entity / domain service layer (`src/services/*`, orchestration)
      → Entity facades (`@/api/entities` — thin `EntityManager` API)
        → Supabase (+ RLS) / optional localStorage mirrors
```

Why the middle layer: one place for **timeouts/retries**, **logging/metrics**, **feature gating**, **error shaping**, and **swapping storage** without rewriting every screen. Hooks stay thin (cache keys, `enabled`, composition); services own **how** data is fetched or mutated.

Anti-pattern: `UI → Invoice.list()` scattered across pages.

Reference example: `useInvoices` → `fetchInvoiceListPage` in **`src/services/InvoiceListService.js`** → `Invoice.list` → `EntityManager` → Supabase. PDF/email/share flows stay in **`src/api/InvoiceService.js`** (different concern: delivery, not list reads).

Canonical profile path (implemented)

Profile state has a single fetch owner:

- **Owner:** `AuthContext` (`src/contexts/AuthContext.impl.jsx`)
- **Consumer path:** `useAuth()` and `useUserProfileQuery()` (selector facade over auth state)
- **Rule:** pages/components should not add new `User.me()` fetches for normal shell state; use auth/context selectors.
- **Allowed exceptions:** explicit one-off snapshot reads in render/send pipelines (PDF generation, email send branding snapshots) where point-in-time profile capture is intentional.

Domain-approved path matrix

Domain	Approved path
Auth/profile	`AuthContext` → `useAuth` / `useUserProfileQuery`
Document lists	hook (`useInvoices`/etc.) → service (`*ListService`) → entity facade
Document exports	page action → `DocumentExportService`
Dashboard bounded data	dashboard hooks → `DashboardDataService`
Side datasets	feature hook with explicit `enabled` gate + stale time

`customClient.js` / EntityManager — online vs offline

`EntityManager` is powerful but easy to misuse: **silent failures**, **localStorage** paths for guests, and **`list()` timeouts** returning an empty cache can look like “no data” when the network failed.

Policy (implemented in `EntityManager`): use **`navigator.onLine`** as a coarse gate:

- **Offline** — do **not** call Supabase for bulk `pullFromSupabase` / empty `find()`; use **in-memory / local** only and **log** (`[Paidly][EntityManager] … offline`). `get()` cache misses throw a **clear** "not available offline" error instead of a failed network round-trip.
- **Online** — **attempt Supabase** as today. When `list()` uses a `maxWaitMs` **race**, the continuation `pull` **promise** logs failures instead of `void pull.catch(() => {})`, and an **empty cache** after the wait emits a **diagnostic warning** (slow vs failed vs still loading).

`skipLocalPersistence` (signed-in + mapped table) already avoids treating **localStorage** as authoritative; the online/offline split further reduces **masked** behaviour.

Session and side effects

Auth session → org scope → the stack above + Query cache → ** `/api/` * for secrets and side effects (email, Payfast, public tokens, admin queues).

B.6 DEPLOYMENT

`vite build` → Vercel static + `vercel.json` rewrites + scheduled crons; production domains alias to the project.

HIGH IMPACT NEXT (PRODUCT — WHAT TO SHIP FIRST)

These three compound **retention**, **differentiation**, and the **business OS** story. Run them as explicit initiatives; the numbered backlog below is hygiene and scale in parallel.

1. Keep the document system honest (split persistence)

- **Invoice + quote + payslip = same engine, different tables** — shared **lifecycle** primitives (draft → send → observe → settle / convert) in `src/document-engine/`, specialised persistence (`invoices`, `quotes`, `payslips`). Do **not** migrate these into `documents`.
- **Documents Hub** — generic business documents only (`documents` / `document_items` / `document_events`).
- **Shared UI** — one **Create / Edit document** shell where it already exists for invoice/quote compose; hub types use typed/dedicated pages. Kind-specific rules stay behind thin adapters.

2. Client Timeline (inside client profile)

A single **chronological timeline** on the client record, for example:

- **Invoice sent** (and viewed / paid where data exists)
- **Quote sent / accepted / declined / expired**
- **Payment received** (linked payment rows)

Data: join `invoices`, `quotes`, `payments`, and optionally `document_sends` / `message_logs` (and later tasks/notes) filtered by `client_id` + `org_id`, sorted by time.

Why it hits: turns "contacts" into **relationship history**—high **retention** and a clear Paidly-only view most invoicing-only tools do not unify on one screen.

3. Conversion flow: Quote → accepted → auto invoice draft

When a quote moves to **accepted** (or explicit user action "Convert to invoice"):

1. **Create** an **invoice draft** (link `quote_id` or metadata for traceability).
2. **Prefill** client, line items, currency, terms/branding from the quote.
3. **Route** the user to **Edit invoice** to review, adjust tax/dates, then send.

This closes the **commercial loop** in-product. Persistence is `quotes` → `invoices`, not the Documents Hub.

Hub job cards, project reports, and scopes **Convert to Invoice / Quote** the same way: specialised compose, specialised tables.

Blueprint PDF: regenerate from this markdown with `npm run docs:blueprint-pdf` (`scripts/generate-blueprint-pdf.mjs`).

FOUNDATION PRIORITIES (RUN WITH HIGH IMPACT)

Ship these in parallel with **High impact next**—they reduce churn and make every subsequent feature cheaper.

4. Stabilize Auth + Session (work already started)

- **Goal:** predictable **sign-in, refresh, tab sync, and logout**; no ghost states where the UI runs but org or profile is wrong.
- **Scope:** Supabase Auth session lifecycle, `getSession` / `getUser` ordering and retries (see patterns in `customClient.js`), `RequireAuth` and role gates, **org bootstrap** (`ensureUserHasOrganization`), **invite / reset-password** flows, **profile** load after cold start.
- **Exit criteria:** short **runbook** (or ADR) for "session failure modes + what the user sees"; fewer uncaught `AbortError` paths; QA checklist for multi-tab and slow network.

5. Standardize UI layout (same structure everywhere)

- **Goal:** one **layout grammar** across the app (see **A.2 — Standardize UI layout** table).
- **Scope:** roll `PageTemplate` through primary lists; align **document editors** on the same shell; admin/settings use `embedded` + `PageHeader` where applicable.
- **Exit criteria:** published **Experience checklist** (one page) that references `PageTemplate` and editor shell; new PRs default to an existing template row—no orphan layouts.

NEXT STEPS (STRATEGY → ENGINEERING BACKLOG)

*Numbering map: **4–5** execute **Foundation §4–5** (auth/session, UI layout)—the same priorities called out in architecture **A.1** and **A.2**. **6** (Experience checklist) **closes** Foundation §5. **7** (role matrix) **supports** Foundation §4. Items **8–11** are the former 6–9 line-up (**PageTemplate** rollout, hook→service pattern, Client Timeline, quote→invoice). Product-led **High impact** items (§1–3) stay in that section above.*

1. **Grow `src/document-engine/`** — status enums, send + PDF adapters, and thin facades over `Invoice` / `Quote` / `Payslip` where behaviour overlaps (**supports § High impact 1**).
2. **Line up quote / invoice / payslip** on the same **deliver + observe** interfaces (even if tables stay separate short term); keep **compose** converging on **Create Document + type**, not parallel product UIs (**supports § High impact 1**).
3. **Add first-class Payment Intent model + APIs** — define intent create/update/reconcile contract between `Document Engine` and `Revenue & Ops` (idempotency, status normalization, expiry/retry rules).
4. **Make Revenue & Ops consumers explicit** — cash flow and reports should pull through document-shaped APIs or views, not ad hoc duplicates (**supports Client Timeline + money story**).
5. **Publish payment UX contract in Experience checklist** — one status vocabulary + CTA matrix for document detail, public invoice, and portal paths.
6. **Stabilize Auth + Session** — execute **Foundation §4** (session/read/write matrix, invites, org bootstrap, documentation).
7. **Standardize UI layout** — execute **Foundation §5** + **A.2** table (`PageTemplate`, editor shell, embedded admin).
8. **Publish an "Experience checklist"** (1 page) for new screens touching documents or money — include the **Page Template** three-zone rule (`PageTemplate`) and **layout grammar** row picker (**closes Foundation §5**).
9. **Identity / role matrix doc** — admin vs management vs support capabilities (complements §4).
10. **Roll out `PageTemplate`** on Invoices, Quotes, Clients, Services, and Affiliates — move filters/summary into `sidePanel` where they still live inside the main card.
11. **Extend the hook → service → entity pattern** — add `*ListService` / query modules for Quotes, Clients, and other high-traffic reads; keep `api/InvoiceService`**-style** modules for non-CRUD delivery concerns.
12. **Implement Client Timeline** — query + UI on **Client detail** per **High impact §2**; consider a small `ClientTimelineService` for aggregation and caching keys.

3. **Implement quote → invoice conversion** — server or client path that creates draft invoice from accepted quote per **High impact §3**; validate RLS and idempotency (no duplicate drafts on double-accept).

Immediate execution order (exact)

1. **Add `payment_intents` table.**
2. **Integrate payment intents into the document lifecycle.**
3. **Expand `document_events` coverage and event ingestion.**
4. **Build payment UI into the sticky right panel on invoice/payment touchpoints.**
5. **Add basic reminders (event- and due-date driven).**

V1 IMPLEMENTATION CONTRACT (DIRECT BUILD PLAN)

Tables (Supabase)

- `payment_intents` (new): `id`, `document_id`, `provider`, `amount`, `currency`, `status`, `external_id`, `created_at`
- `document_events` (expand): ensure canonical events `sent`, `opened`, `clicked`, `paid`, `reminded` with `occurred_at`, `actor_type`, `metadata`
- `payments` (existing): keep as settlement/reconciliation records linked to provider/webhook outcomes

Services (app/server orchestration)

- `DocumentPaymentIntentService` (new): create/update intent from document snapshot; enforce idempotency by `document_id + provider + status window`
- `DocumentEventService` (expand): append normalized document/payment lifecycle events; provide query helpers for timeline/reminders
- `PaymentWebhookReconciliationService` (new or expanded from current Payfast handler): verify provider payloads, map to canonical statuses, write `payments` + `document_events`, trigger settle transition
- `PaymentIntelligenceService` (v1 basic): evaluate reminder triggers (e.g., viewed-not-paid, due/overdue windows) and emit follow-up actions

Cron jobs (Vercel)

- `POST /api/cron/reminders` (existing pattern; expand logic): process due/overdue + behavior-based reminder candidates
- `POST /api/cron/payment-intelligence` (new): run lightweight trigger evaluation (`viewed>=N && not paid`, retry windows)
- `POST /api/cron/subscriptions-dunning` (existing/adjacent): keep tenant billing and subscription retries isolated from document settlement logic

API endpoints (`/api/*`)

- `POST /api/payment-intents` (new): create intent for payable document
- `GET /api/payment-intents/:id` (new): fetch intent status for sticky panel + public views
- `POST /api/payments/webhook/:provider` (new canonical route; may proxy existing `payfast-handler`)
- `POST /api/pos/webhook/:token` (POS sale ingress — generic, Yoco, Square parsers)
- `POST /api/pos/webhook/provider/square` (Square application webhook — routes by merchant_id)
- `POST /api/pos/oauth/square/start` · `GET /api/pos/oauth/callback/square` (Square OAuth connect)
- `POST /api/pos/oauth/yoco/connect` (Yoco API key connect + auto webhook)
- `GET|POST|PATCH|DELETE /api/pos/connections` (org settings managers / company admins); RLS mirrors this via `is_company_admin_for_org` (members may SELECT only)
- `GET /api/pos/sales` (org members — dashboard read model)
- `POST /api/documents/:type/:id/events` (new internal endpoint) or service-only ingestion path for `document_events`
- `POST /api/reminders/dispatch` (new internal endpoint used by cron workers)

v1 acceptance checks:

- Invoice can move `Deliver → Observe → Payment Intent → Settle` using provider-verified events only.
- Sticky right panel shows intent-aware CTA states (`Pay now`, `Retry payment`, `Send reminder`) without page-specific logic forks.
- `document_events` powers timeline entries and at least one automated reminder trigger.

End of document.